

Homework 15: A Contact Book

Build a lookup API, prove the `orElse` trap with a counter you can watch, and fix four anti-patterns.

The Data

```
record Contact(String name, String email, String phoneOrNull) { }

Map.of(
    "ada",    new Contact("ada",    "ada@example.com",    "555-0100"),
    "grace",  new Contact("grace",  "grace@example.com", null),
    "alan",   new Contact("alan",   "alan@example.com",   "555-0199"));
```

Note that `grace` has no phone, and the field is called `phoneOrNull` for a reason you will implement in part three.

Part One: Lookups Without Unwrapping

Create `ContactBook.java` in the classic form.

Write `find(String name)` returning `Optional<Contact>`.

Then, for `ada`, `grace` and `nobody`, print each contact's phone number and their email domain. Missing values get a readable fallback.

Rules:

- No `isPresent` anywhere in this part
- No `get` anywhere in the whole file
- The phone lookup must need `flatMap`, not `map`. Show both and say why the `map` version does not even compile into an `orElse(String)`.

Then chain three fallbacks with `or()` and explain in a comment why `orElse` could not have been used to build that chain.

Finish by mapping a list of names through `find` and collecting only the hits, using `Optional::stream` rather than a filter.

Part Two: Prove The Trap

Add a `static int defaultLookups` counter and a `lookupDefaultEmail()` method that increments it.

Then run three cases and print the counter after each:

Case	Expect
<code>orElse(lookupDefaultEmail())</code> on a present Optional	?
<code>orElseGet(ContactBook::lookupDefaultEmail)</code> on the same	?
<code>orElseGet(...)</code> on an empty Optional	?

Write down what you predicted before running it, then what actually happened.

Explain It

In a comment, in your own words:

1. Why the argument to `orElse` runs at all when the value is present
2. Why `orElseGet` does not
3. **Why this is a correctness bug and not just a performance note.** Your answer must refer to the counter, and say what it would be in real code.

Part Three: Fix Four Anti-Patterns

Show the fixed form of each, with a comment naming what was wrong:

1. `isPresent` then `get` — replace with a single chained call
2. `Optional` as a field — keep `phoneOrNull` plain, return `Optional` from the accessor, and say what an `Optional` field would have cost
3. `Optional` as a parameter — write a method taking an optional label using an **overload** instead
4. `Optional<List<T>>` — write a search returning an empty list, and say how many states the wrapped version would have forced on the caller

Then replace a `get()` with `orElseThrow` and a message. Print both that message and what `get()` would have said, and comment on the difference.

Part Four: What It Does Not Protect You From

Write a method declared to return `Optional<String>` that returns `null`.

Call it, print what came back, then call any method on the result and catch what happens.

In a comment, explain why this is worse than a plain nullable return, and why it proves `Optional` is a convention rather than a guarantee. Compare with Kotlin or Rust, where the compiler is involved.

Acceptance Criteria

- ☐ Runs with `java ContactBook.java`

- ☐ `get()` appears nowhere in the file
- ☐ `isPresent` appears nowhere except where you are demonstrating the anti-pattern
- ☐ The phone lookup uses `flatMap`, with both forms shown
- ☐ `or()` chains three fallbacks, with a comment on why `orElse` cannot
- ☐ `Optional::stream` is used to drop misses
- ☐ The counter prints three times with the three different results
- ☐ The explanation covers argument evaluation and why it is a correctness bug
- ☐ All four anti-patterns are shown fixed, each with a comment
- ☐ `orElseThrow`'s message is compared with `get()`'s
- ☐ The null-returning `Optional` method is demonstrated and explained

Stretch

Write `Optional<Contact> findAny(String... names)` returning the first name that exists, evaluating no more lookups than necessary.

Then write a comment on whether your version is lazy, and if you used a stream, what `findFirst` did that a loop over `or()` would not have.

Hint, if part one's `flatMap` point is not landing

Try writing `find("ada").map(Contact::phone).orElse("none")` and read the compiler error. The type is `Optional<Optional<String>>`, and `orElse` on that wants an `Optional<String>`, not a `String`. The error is the lesson.